



Cairo University
Faculty of Engineering
Department of Computer Engineering

Accelera



A Graduation Project Report Submitted
to
Faculty of Engineering, Cairo University
in Partial Fulfillment of the Requirements for the Degree
of
Bachelor of Science in Computer Engineering.

Presented by

Mohamed Ashraf
Mazen Adel

Nesma Osama
Mohamed Khater

Supervised by

Dr. Salma Abdel Monem

April 17, 2026

All rights reserved. This report may not be reproduced in whole or in part, by photocopying or other means, without the permission of the authors or department.

Abstract

Accelerera is a hybrid Python/C++ machine learning framework that combines a high-level Python interface with native execution components. The system is organized around five modules: Accelerera Pipe, the Code Parallelizer, AutoPreprocessing, AutoML, Deployment, and the Benchmark Platform. Accelerera Pipe represents machine-learning workflows as directed graph nodes for preprocessing, modelling, prediction, metric evaluation, branching, and merging. The Code Parallelizer analyzes C/C++ and supported Python code, extracts loop features, predicts parallelization opportunities, and emits OpenMP-annotated C++ code. This report documents the design, methodology, and experimental results available for the implemented modules, while retaining the required report structure for the remaining modules.

Arabic Abstract

Acknowledgment

Table of Contents

Abstract	i
Arabic Abstract	ii
Acknowledgment	iii
List of Abbreviations	x
List of Symbols	xi
1 Introduction	1
1.1 Motivation and Justification	1
1.2 Project Objectives and Problem Definition	1
1.3 Project Outcomes	1
1.4 Document Organization	1
2 Market Visibility Study	2
2.1 Targeted Customers	2
2.2 Market Survey	2
2.2.1 AutoCleanML	2
2.2.2 Kaggle	2
2.3 Business Case and Financial Analysis	3
3 Literature Survey	4
3.1 Background and Previous Work	4
3.1.1 Data Science	4
3.1.2 Machine Learning	4
3.1.3 Supervised Machine Learning	4
3.1.4 Classification	4
3.1.5 Regression	4
3.1.6 Natural Language Processing NLP	4
3.1.7 Exploratory Data Analysis	5
3.1.8 Data Preprocessing	5
3.1.9 Missing Value Imputation	5
3.1.10 Robust Scaling	5
3.1.11 Categorical Encoding	5

3.1.12 Binary Encoding	5
3.1.13 Frequency Encoding	6
3.1.14 Term Frequency-Inverse Document Frequency TF-IDF	6
3.1.15 Data augmentation	6
3.1.16 Classification Evaluation Metrics	6
3.1.16.1 Accuracy	6
3.1.16.2 Precision	7
3.1.16.3 Recall	7
3.1.16.4 F1-score	7
3.1.17 Regression Evaluation Metrics	7
3.1.17.1 Mean Absolute Error	7
3.1.17.2 Mean Squared Error	7
3.1.17.3 R-squared Score	7
3.2 Comparative Study of Previous Work	8
3.2.1 Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices [1]	8
3.3 Implemented Approach	8
3.3.1 Auto Preprocessing	8
4 System Design and Architecture	10
4.1 Overview and Assumptions	10
4.1.1 Accelera Pipeline Assumptions	10
4.1.2 Auto Preprocessing Assumptions	10
4.1.2.1 Tabular Dataset Assumptions	10
4.1.2.2 Text Dataset Assumptions	10
4.1.2.3 Image Dataset Assumptions	11
4.1.3 Benchmark Assumptions	11
4.2 System Architecture	12
4.2.1 Block Diagram	12
4.3 Accelera Pipe Module	13
4.3.1 Functional Description	13
4.3.2 Modular Decomposition	13
4.3.3 Design Constraints	16
4.3.4 Other Description of Module	17
4.4 Code Parallelizer Module	18
4.4.1 Functional Description	18
4.4.2 Modular Decomposition	18
4.4.3 Design Constraints	20
4.4.4 Other Description of Module	21
4.5 AutoPreprocessing Module	22
4.5.1 Functional Description	22
4.5.2 Modular Decomposition	23

4.5.2.1	Input and Configuration Validation	23
4.5.2.2	Data Understanding and Reporting	23
4.5.2.3	Tabular Preprocessing	23
4.5.2.4	Target Preprocessing	23
4.5.2.5	Feature Selection	24
4.5.2.6	Text Preprocessing	24
4.5.2.7	Image Preprocessing	24
4.5.2.8	Artifact Saving	24
4.5.3	Design Constraints	24
4.5.4	Other Description of the Module	25
4.6	AutoML Module	25
4.7	Deployment Module	25
4.8	Benchmark Platform Module	25
4.8.1	Functional Description	25
4.8.2	Modular Decomposition	26
4.8.2.1	Benchmark Management	26
4.8.2.2	Metric Management	26
4.8.2.3	Submission Management	26
4.8.2.4	Validation and Scoring Scripts	26
4.8.2.5	Leaderboard Interface	26
4.8.3	Design Constraints	27
4.8.4	Other Description of the Module	27
5	System Testing and Verification	28
5.1	Testing Setup	28
5.2	Testing Plan and Strategy	28
5.3	Accelera Pipe Module	28
5.4	Code Parallelizer Module	30
5.5	AutoPreprocessing Module	33
5.5.1	Experimental Setup	33
5.6	AutoML Module	36
5.7	Deployment Module	36
5.8	Benchmark Platform Module	36
6	Conclusions and Future Work	37
6.1	Faced Challenges	37
6.2	Gained Experience	37
6.3	Conclusions	37
6.4	Future Work	37
	References	38
A	Development Platforms and Tools	40

B Use Cases	41
C User Guide	42
D Code Documentation	43
E Feasibility Study	44

List of Figures

4.1	Before duplicate first-node sharing	16
4.2	After duplicate first-node sharing	16
4.3	Code Parallelizer module workflow	19
4.4	Integration flow for compiled preprocessing inside Accelera Pipe	21
5.1	Classification Performance Comparison (F1-Macro Score)	35
5.2	Classification Preprocessing Time Comparison	35
5.3	Regression Performance Comparison (R^2 Score)	36
5.4	Regression Preprocessing Time Comparison	36

List of Tables

- 4.1 Main components of the Accelera Pipe module 14
- 4.2 Mapping from builder calls to graph node semantics 15
- 4.3 Main components of the Code Parallelizer module 19

- 5.1 Accelera Pipe latency scaling in seconds 28
- 5.2 Accelera Pipe RSS memory scaling in MB 29
- 5.3 Selected candidate and accuracy across sample sizes 29
- 5.4 Largest Accelera Pipe comparison 30
- 5.5 OpenMP classifier report 30
- 5.6 Hard parallelizer evaluation 31
- 5.7 Linux parallelizer and Accelera Pipe integration benchmark 31
- 5.8 Linux direct example-script verification runs 32
- 5.9 Windows direct example correctness and path timing 32
- 5.10 Windows compilation and process overhead 32
- 5.11 Summary of Datasets Used in Experiments 34

List of Abbreviations

AI	Artificial Intelligence
AST	Abstract Syntax Tree
CPU	Central Processing Unit
ML	Machine Learning
OpenMP	Open Multi-Processing
RSS	Resident Set Size
SVC	Support Vector Classifier
VMS	Virtual Memory Size

List of Symbols

X	Input feature matrix
y	Target vector
P	Source program
L	Extracted loop
$F(L)$	Feature representation of loop L

Contacts

Team Members

Name	Email	Phone Number
Name of Student 1	abc1@email.com	+2 01xxxxxxxxx
Nesma Osama	nesma.ahmed03@eng-st.cu.edu.eg	+2 01067193062
Name of Student 3	abc1@email.com	+2 01xxxxxxxxx
Name of Student 4	abc1@email.com	+2 01xxxxxxxxx

Supervisor

Name	Email	Number
Salma Abdelmoniem	abc1@email.com	+2 01114302362

Chapter 1: Introduction

1.1 Motivation and Justification

Accelerera addresses the practical gap between the productivity of Python and the performance requirements of machine-learning workflows and loop-heavy data processing. A sequential script evaluates alternatives one after another and leaves memory lifetime and parallel execution decisions to the user. The project provides a graph-based pipeline backend and a code parallelization path to make these execution decisions explicit and reusable.

1.2 Project Objectives and Problem Definition

The project objective is to provide an integrated framework for constructing, executing, evaluating, and accelerating data-science workflows. The problem is to preserve a convenient Python-level programming model while enabling native graph scheduling, controlled intermediate-data lifetime, and OpenMP-backed execution for supported custom code.

1.3 Project Outcomes

The project is organized into the following modules:

- Accelerera Pipe for graph-based machine-learning workflows.
- Code Parallelizer for source analysis and OpenMP generation.
- AutoPreprocessing for tabular, text, and image data preparation.
- AutoML for model selection and optimization.
- Deployment and Benchmark Platform components for serving and evaluation.

1.4 Document Organization

Chapter 2 reserves the required market visibility study. Chapter 3 contains the literature-survey structure. Chapter 4 documents the system design and implemented module methodology. Chapter 5 presents testing and verification results. Chapter 6 concludes the report and records future work. The appendices retain the required development, user-guide, and feasibility-study sections.

Chapter 2: Market Visibility Study

In this chapter, we'll identify Accelera's target customers, talk about similar tools and platforms in the market, discuss their pros and cons, and provide the business case study and financial analysis of the system.

2.1 Targeted Customers

Our targeted customers are data scientists and machine learning engineers, both beginners and experts. Accelera aims to support these users by providing a tool that automates and accelerates several steps in the data science life cycle. This helps users reduce repetitive manual work, and save time.

2.2 Market Survey

We found that many existing platforms provide useful features for data science and machine learning development, but these features are often available in separate tools or are limited to specific parts of the workflow. However, Accelera aims to provide these features within one toolkit.

2.2.1 AutoCleanML

It is a Python tool mainly focused on automating data cleaning and preprocessing tasks for machine learning datasets.

Pros:

- Makes machine learning data preparation automatic and smart.
- Provides a report that explains the preprocessing decisions made by the tool.

Cons:

- Only designed for cleaning and preparing data..
- It does not support preprocessing for image and text datasets.

2.2.2 Kaggle

Kaggle is an online platform where the data science and machine learning community comes together. It is best known for its competition platform.

Pros:

- It introduces competitions.

- It ranks users in competitions.
- It has many public datasets.

Cons:

- It mainly depends on the notebooks style, so projects can get hard to organize.
- It doesn't provide a complete machine learning workflow in a single toolkit.

2.3 Business Case and Financial Analysis

Chapter 3: Literature Survey

In this chapter, we give an overview of the topics, algorithms, and techniques needed to understand the project as a whole. We discuss the technical subjects and background knowledge, and then summarize the research papers related to our project.

3.1 Background and Previous Work

This section explains the concepts and techniques used in Accelerera units.

3.1.1 Data Science

Data science is a field that extracts knowledge and insights from data using scientific methods, processes, and algorithms.

3.1.2 Machine Learning

Machine learning is a field of artificial intelligence that lets computers learn patterns from data without needing to be explicitly programmed. A machine learning model is trained using training data , and then it uses what it learned to make predictions on new data.

3.1.3 Supervised Machine Learning

The training dataset includes a label column that represents the target output. This column is used during training to learn the mapping between the input features and the expected result.

3.1.4 Classification

Classification is a supervised machine learning task whose output is a label or category.

3.1.5 Regression

Regression is a supervised machine learning task whose output is a continuous numerical value.

3.1.6 Natural Language Processing NLP

Natural language processing is a field that focuses on enabling computers to understand and work with human language. Text data can't be used directly by most machine learning models, so it needs to be cleaned and turned into numerical features

first.

3.1.7 Exploratory Data Analysis

Exploratory Data Analysis is an important step in data analysis where we explore, summarize, and visualize data to understand its structure, detect patterns, and check relationships between variables.

3.1.8 Data Preprocessing

Data preprocessing is the stage of transforming raw data into a clean and usable format for machine learning models.

3.1.9 Missing Value Imputation

Missing value imputation is a preprocessing technique used when some values in a dataset are empty. Instead of removing all rows that contain missing values, the missing values can be replaced using a suitable value.

3.1.10 Robust Scaling

Robust scaling is a scaling technique that uses the median and the interquartile range instead of the mean and standard deviation.

$$x' = \frac{x - \text{median}(X)}{\text{IQR}(X)}$$

where x' is the scaled value, x is the original value, $\text{median}(X)$ is the median of the feature values, and $\text{IQR}(X)$ is the interquartile range.

3.1.11 Categorical Encoding

Categorical encoding is the process of converting text categories into numeric values. Many traditional machine learning models and preprocessing pipelines expect numerical input.

3.1.12 Binary Encoding

Binary encoding is a categorical encoding technique that first gives each category a number, then converts this number into binary form. The binary digits are then stored as separate columns.

3.1.13 Frequency Encoding

Frequency encoding replaces each category with how often it appears in the dataset. If a category appears many times, it receives a higher frequency value.

$$f(c) = \frac{\text{count}(c)}{N}$$

where $f(c)$ is the frequency of category c , $\text{count}(c)$ is the number of rows containing this category, and N is the total number of rows.

3.1.14 Term Frequency-Inverse Document Frequency TF-IDF

It is a technique used to convert text into numerical features by giving each word a weight based on how important it is in a document and across the whole dataset. Term Frequency measures how often a word appears in a document. Inverse Document Frequency gives lower weight to words that appear in many documents, because these words are usually less useful for classification.

$$TFIDF(t, d) = TF(t, d) \times IDF(t)$$

where $TFIDF(t, d)$ is the weight of term t in document d , $TF(t, d)$ is the frequency of the term in the document, and $IDF(t)$ measures how rare the term is across all documents.

3.1.15 Data augmentation

Is a technique used to increase diversity of a dataset without actually collecting new data. It works by applying various transformations to the existing data to create new, modified versions of data that helps the model generalize better.

3.1.16 Classification Evaluation Metrics

Classification metrics are used to evaluate the performance of a classification model.

3.1.16.1 Accuracy

Accuracy measures how many predictions are correct out of all predictions:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP is true positives, TN is true negatives, FP is false positives, and FN is false negatives.

3.1.16.2 Precision

Precision measures how many predicted positive samples are actually positive:

$$Precision = \frac{TP}{TP + FP}$$

3.1.16.3 Recall

Recall measures how many actual positive samples were correctly detected:

$$Recall = \frac{TP}{TP + FN}$$

3.1.16.4 F1-score

The F1-score combines precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

3.1.17 Regression Evaluation Metrics

Regression metrics are used when the model predicts continuous numerical values.

3.1.17.1 Mean Absolute Error

Mean Absolute Error measures the average absolute difference between the real values and predicted values:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

where y_i is the actual value and $\hat{y}_i$ is the predicted value.

3.1.17.2 Mean Squared Error

Mean Squared Error gives more penalty to large errors:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

3.1.17.3 R-squared Score

R-squared score measures how well the regression model explains the variation in the actual values.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Where y_i is the actual value, $\hat{y}_i$ is the predicted value, and $\bar{y}$ is the mean of the actual values.

3.2 Comparative Study of Previous Work

3.2.1 Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices [1]

This paper shows the role of data preprocessing and feature engineering in machine learning and data mining projects. It says that preprocessing is not only a preparation step before model training, but also an important part of the complete machine learning pipeline. It also presents a general preprocessing framework that includes data cleaning, data transformation, feature construction, feature selection, and dimensionality reduction. For each stage, it explains different techniques and discusses the advantages and disadvantages of each one.

3.3 Implemented Approach

3.3.1 Auto Preprocessing

In this module we decided to implement an automated preprocessing approach for tabular datasets. The approach was designed to prepare data for machine learning models by applying suitable preprocessing steps automatically according to the type and characteristics of each column and some user configurations.

The general framework of the approach is as follows:

1. Take the dataset and some user configurations.
2. Remove duplicate records.
3. Split the data into training and validation sets.
4. For each column, we get this information: data type, percentage of missing values, unique values, and basic stats.
5. Drop unnecessary columns, such as constant columns or columns with missing values above the selected threshold.
6. Detect column types automatically.
7. Make Exploratory Data Analysis.
8. Apply missing value imputation.
9. Apply categorical encoding.
10. Apply robust scaling.
11. Apply feature selection using mutual information.

12. Automatically saves all needed files for inference.
13. Generate a report.

The chosen approach follows a classical automated preprocessing pipeline. The main preprocessing decisions are:

- **Missing values:** Median imputation is used for numerical features because it is simple and more robust to outliers than mean imputation. For categorical features, most-frequent imputation is used because it replaces missing values with valid existing categories.
- **Numerical scaling:** `RobustScaler` is used for numerical features because tabular datasets may contain outliers or skewed values. It depends on the median and interquartile range, which makes it more stable than standard scaling when extreme values exist.
- **Categorical encoding:** The encoding method is selected according to the feature type and cardinality. Binary columns are encoded using ordinal encoding, low-cardinality categorical columns are encoded using binary encoding, and high-cardinality categorical columns are encoded using frequency encoding. This helps reduce the number of generated features and avoids high dimensionality.
- **Feature selection:** Mutual information is used because it is a filter-based method that can work with both classification and regression tasks. It helps keep the most informative features and remove weak features before model training.

This approach was chosen because it gives a good balance between automation, simplicity, and speed. More advanced techniques, such as K-nearest neighbour imputation, autoencoders, and wrapper feature selection, can be useful in specific cases, but they need more computation.

One of the important additions in this project is that the module is not only used for data preparation. It also supports exploratory data analysis report.

Also the module was extended to support text and image data preparation, not only tabular data.

Chapter 4: System Design and Architecture

4.1 Overview and Assumptions

This section introduces the assumptions made for each module in Accelera and explains why these assumptions were needed during the implementation.

4.1.1 Accelera Pipeline Assumptions

The system combines Python-facing APIs with native execution components. The design assumes that machine-learning workflows can be represented as a graph of operations and that only a restricted, analyzable subset of custom source code should be translated to parallel C++.

4.1.2 Auto Preprocessing Assumptions

For the AutoPreprocessing module, the assumptions depend on the type of dataset being processed. The module supports three types of data: tabular, text, and image datasets.

4.1.2.1 Tabular Dataset Assumptions

For tabular datasets, the AutoPreprocessing module assumes that:

- The dataset consists of numerical and categorical features, and the task is either classification or regression. Because we try to implement the standard structured machine learning problems where supervised learning is applied..
- The user provides configuration parameters to control preprocessing operations. This allows the pipeline to adapt to different datasets and user requirements.

4.1.2.2 Text Dataset Assumptions

For text datasets, the AutoPreprocessing module assumes that:

- The dataset contains a single text column and is intended for a text classification task with a defined target label. Because This simplifies the pipeline to standard supervised NLP classification problems and prevents the feature space from becoming very large after transforming the text into a numerical representation, making it suitable for classical machine learning techniques.

- The user provides configuration parameters for text cleaning and feature extraction. To give the user a level of control over the preprocessing pipeline, such as limiting the maximum number of numerical features generated after transforming the text into a numerical representation.

4.1.2.3 Image Dataset Assumptions

For image datasets, the AutoPreprocessing module assumes that:

- The dataset is intended for image classification and follows a folder-based class structure. Because there are many vision tasks, but we implemented the most common one to apply its processes.
- All images are stored in supported formats and are accessible. This ensures compatibility with image loading and preprocessing libraries.
- The user provides configuration parameters for preprocessing and augmentation. Because different datasets require different preprocessing strategies such as resizing, normalization, and augmentation.

The dataset is expected to follow the directory structure shown below, where the validation folder is optional:

```
Training/
  Cats/
  Dogs/
```

```
Validation/
  Cats/
  Dogs/
```

4.1.3 Benchmark Assumptions

For the Benchmark module, the following assumptions are made. These assumptions are introduced due to system design constraints, particularly the limited server storage capacity for handling large dataset uploads, as well as to ensure fair and consistent evaluation and comparison between users.

- When the user creates a benchmark, they provide a tabular dataset. This ensures that the module operates on structured data suitable for standard machine learning evaluation.
- All datasets are stored in publicly accessible Google Drive links. This design choice is adopted to reduce server storage requirements.
- The user must provide three CSV files:

- A training dataset containing both features and target labels.
- A testing dataset containing the same feature structure as the training set but without the target column.
- A ground truth target file containing only the target labels for the test set.

This separation ensures a standard evaluation protocol and prevents data leakage during benchmarking.

- When submitting predictions, the user provides a Google Drive link to a CSV file containing the predicted results. This ensures consistency in submission format across all users.
- The submitted prediction file must have the same length and format as the expected output to ensure correct evaluation and fair comparison between different users' models.
- When creating a benchmark, it is assumed that the user selects an evaluation metric that is appropriate for the problem type and provides a correct metric configuration. This is necessary to guarantee meaningful and valid performance measurement.
- For admin users, when defining an evaluation metric, it is assumed that a valid metric from the Scikit-learn library is selected. Because we use Scikit-learn metrics for evaluation.
- The selected metric must be properly configured and is expected to return a single scalar value for performance comparison and ranking purposes.
- The user is allowed to have only one active submission per benchmark. The user cannot create multiple submissions, but they are allowed to update their existing submission. This constraint is enforced to reduce storage usage.

4.2 System Architecture

4.2.1 Block Diagram

The implemented system is organized around graph execution, source-code parallelization, automated preprocessing, model-selection, deployment, and benchmarking components. The following module sections describe the material available in the previous thesis.

4.3 Accelera Pipe Module

4.3.1 Functional Description

The Accelera Pipe module is the high-level workflow construction layer of the system. Its main function is to let the user describe a machine-learning pipeline in Python while delegating the actual graph construction, scheduling, branch execution, and runtime data management to the native C++ backend. Instead of forcing the user to manually manage intermediate outputs between preprocessing, modelling, prediction, and evaluation stages, the module exposes a fluent builder interface where every call adds a new computational node to an internal directed acyclic graph (DAG).

From the user's point of view, the module behaves like a pipeline object. A user can add preprocessing steps, models, prediction nodes, metrics, merge operations, and alternative branches. Internally, however, these operations are not stored as a simple linear list. Each operation is converted into a graph node with a specific type such as `PREPROCESS`, `MODEL`, `PREDICT`, `MERGE`, or `METRIC`. This representation is important because it allows the same pipeline API to support both sequential workflows and branched workflows where multiple candidate paths can be evaluated in parallel.

The module also separates training-time execution from inference-time execution. When a pipeline is called, the native graph executes all required nodes and returns two categories of output: the computed predictions or metric results, and an `ExecutedGraph` object. The `ExecutedGraph` contains only the selected fitted execution path and can be reused later for inference. This is a practical design decision because training normally requires validation data, metric objects, temporary transformed arrays, and candidate branches, while inference should retain only the fitted transformations and models required to process new input data.

Another functional role of Accelera Pipe is transparent optimization of custom preprocessing logic. When the user provides a custom preprocessing function or a custom transformer, the pipeline sends it through the Code Parallelizer before storing it in the graph. Therefore, the user can write a normal Python preprocessing function, while the system attempts to replace loop-heavy parts with a compiled Python-callable C++ implementation when the transformation is safe and supported. If the optimization cannot be applied, the original function is kept, so the external behavior of the pipeline remains unchanged.

4.3.2 Modular Decomposition

The Accelera Pipe module is decomposed into a Python API layer and a native C++ graph execution layer. The Python layer is responsible for usability, builder semantics, metric wrapping, serialization calls, and integration with the Code Parallelizer. The C++ layer is responsible for graph storage, node connections, execution order, parallel scheduling, selected-path extraction, and memory lifetime management.

Table 4.1: Main components of the Accelera Pipe module

Component	Responsibility
<code>core/pipeline.py</code>	Implements the public builder API. It exposes <code>preprocess</code> , <code>model</code> , <code>predict</code> , <code>metric</code> , <code>merge</code> , and <code>branch</code> . It also calls the native graph execution method and wraps the selected fitted path into an <code>ExecutedGraph</code> .
<code>core/pipeline_base.py</code>	Owns the native <code>graph.Graph</code> object, enables parallel graph execution by default, maps Python node names to C++ <code>NodeType</code> values, and provides save/load behavior using pickle.
<code>core/executed_graph.py</code>	Represents the reusable inference graph produced after training. It executes the selected fitted path on new data and can temporarily enable metrics when <code>y_true</code> is supplied.
<code>wrappers/</code>	Contains metric and display wrappers that adapt supervised and unsupervised metrics to the graph execution format.
<code>include/core/graph.hpp</code> and <code>src/core/graph.cpp</code>	Define and implement the native graph object. This layer stores nodes, computes execution order, runs nodes sequentially or in parallel, selects the best branch, and serializes graph state.
<code>include/core/node.hpp</code> and node implementations	Define the common node abstraction and specialized node behavior for input, preprocessing, model, prediction, merge, and metric stages.

At the Python API level, each builder method creates or configures one graph operation. A preprocessing node stores a function or transformer together with execution parameters such as `execute_fit` and `cache`. Before a preprocessing function is stored, the module attempts to parallelize it through `parallelizer.parallelize`. Model nodes store estimators, prediction nodes store test data and the prediction method name, metric nodes store metric wrappers, and merge nodes store the merge strategy.

Table 4.2: Mapping from builder calls to graph node semantics

Builder method	Node type	Main role
<code>preprocess(name, func)</code>	PREPROCESS	Transform input data or selected columns
<code>model(name, model)</code>	MODEL	Fit or apply a machine-learning estimator
<code>predict(name, test_data)</code>	PREDICT	Run a model output function such as <code>predict</code>
<code>metric(name, metric)</code>	METRIC	Evaluate predictions using a supervised or unsupervised metric
<code>merge(name, strategy)</code>	MERGE	Combine outputs from multiple prediction branches
<code>branch(name, ...)</code>	Multiple nodes	Create alternative candidate paths in the DAG

The native backend uses a graph abstraction rather than a linear pipeline. The Graph class stores all nodes, tracks whether the graph has already been compiled, supports cloning, and exposes methods for adding nodes, splitting branches, executing, clearing, saving, loading, and enabling parallel execution. The graph computes a topological execution order before running. This guarantees that each node is executed only after its source nodes have produced their outputs. If the graph contains branches, the execution result is followed by path selection using one of the supported strategies: maximum metric, minimum metric, custom strategy, or all paths.

Algorithm 4.1: High-level Accelera Pipe training flow

Input: training data X, optional labels y, graph G, selection strategy S

Output: predictions or metric results R, executed graph E

- 1: Add X and y to the input node of G
- 2: Compile G if its topological order is not already available
- 3: Execute graph nodes sequentially or in parallel depending on G settings
- 4: Collect outputs from final leaf nodes
- 5: If G contains branches, select the path requested by S
- 6: Clone the selected fitted path into a compact executed graph E
- 7: Remove validation-only data and temporary training data
- 8: Return R and E

Branching is handled by the native `split` operation. Each branch is converted into a chain of graph nodes starting from the current leaf. The backend uses the declared node type and the stored Python object to instantiate each branch node. A branch may contain a single node or a list of nodes, so the same mechanism supports simple model comparison as well as full alternative sub-pipelines.

A useful optimization in branch construction is duplicate first-node sharing. If several branches begin with the same first node, the graph creates that first node once and connects the later branch consumers to its output. This is safe for the first node because all such branches receive the same original input. The optimization is deliberately conservative: later repeated nodes are not merged automatically because their inputs may already differ due to earlier branch operations.

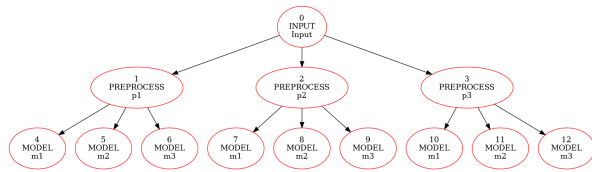


Figure 4.1: Before duplicate first-node sharing

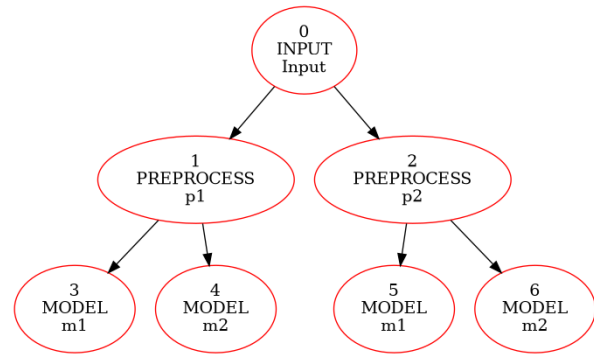


Figure 4.2: After duplicate first-node sharing

4.3.3 Design Constraints

The first design constraint is that the graph must remain a valid DAG. Pipeline execution depends on topological sorting, so cyclic dependencies cannot be allowed. Node connections are therefore restricted by node type. For example, a preprocessing node may follow the input node or another preprocessing node, a model node may follow input or preprocessing output, a prediction node must follow a model, a metric node must follow a prediction or merge node, and a merge node combines prediction outputs. These connection rules protect the execution engine from invalid graphs and preserve the semantic meaning of every stage.

The second constraint is safe memory management. Machine-learning pipelines can produce large intermediate arrays, especially when several preprocessing branches are evaluated. Accelera Pipe therefore tracks remaining consumers for each node output. Once the last downstream consumer finishes, the corresponding intermediate data can be released. Model and metric nodes are treated more carefully because they may hold fitted estimators or evaluation state, while input and preprocessing data can often be cleared after their consumers are done. This policy reduces memory pressure without changing the logical result of the graph.

Algorithm 4.2: Consumer-count based intermediate release

Input: executed node N , remaining consumer table C

Output: graph with unused temporary data released

```

1: for each source node  $S$  that feeds  $N$  do
2:    $C[S] = C[S] - 1$ 
3:   if  $C[S] == 0$  and  $S$  is releasable then
4:     clear the temporary data stored by  $S$ 
5:   end if
6: end for

```

The third constraint is safe handling of mutable preprocessing input. Some transformers may mutate the input array in place, while many scikit-learn transformers return a transformed array without modifying the original input. The backend therefore

includes an inspection step that avoids unnecessary copies for ordinary scikit-learn objects but remains conservative for custom functions or objects that explicitly disable copying. This balances performance with correctness: unnecessary copying is avoided when possible, but unsafe aliasing is not assumed away.

The fourth constraint is parallel execution safety. The graph can run independent nodes in parallel, but a node can start only after all its parents have finished. The backend uses CPU thread semaphores, a separate GPU semaphore, and dependency checks over the topological order. This prevents a downstream node from reading incomplete data and also serializes GPU work when necessary. The number of CPU threads is based on hardware concurrency but can be overridden using the `ACCELERATOR_NUM_THREADS` environment variable.

Finally, the executed graph must not retain training-only data. After training, the selected path is cloned, validation data stored inside prediction nodes is cleared, metric target values are removed, and temporary arrays in the training graph are released. This constraint is essential because the executed graph is intended to be saved or reused for inference, not to duplicate the whole training runtime state.

4.3.4 Other Description of Module

The Accelera Pipe module also supports persistence. The Python `PipelineBase` class saves and loads pipeline objects using pickle. The C++ graph exposes its state as a Python dictionary containing node information, selected-path flags, source indices, cached data, graph compilation state, and parallel execution configuration. During loading, the graph rebuilds the nodes and reconnects them using the stored source indices. This design makes the graph serializable even though part of the implementation is native C++.

The module can also export a graph-like representation of the pipeline structure. The graph utility code writes an XML-style network description containing layers for input, preprocessing, model, prediction, merge, and metric nodes, together with the edges between them. This representation is useful for debugging and for showing how the high-level pipeline has been lowered into an executable graph.

In summary, Accelera Pipe is not only a convenience wrapper over scikit-learn objects. It is a hybrid Python/C++ pipeline execution system. Python provides the user-facing construction interface, while C++ provides the graph semantics, execution scheduling, branch selection, and memory-lifetime control. This split is the main reason the module can remain easy to use while still supporting parallel execution and advanced graph-level optimizations.

4.4 Code Parallelizer Module

4.4.1 Functional Description

The Code Parallelizer module is responsible for transforming supported loop-heavy source code into optimized C++ code annotated with OpenMP pragmas. It is designed to reduce the execution time of custom preprocessing functions and standalone code snippets by identifying loops that can be safely executed in parallel. The module accepts different input forms: a file path, a source string, a custom Python function, or a custom transformer instance. Regardless of the input form, the scientific goal is the same: normalize the input into a C++ loop analysis path, detect parallelization opportunities, and return either parallelized source code or a compiled Python-callable native function.

For Python input, the module first lowers a restricted Python subset into C++. This conversion is intentionally limited to predictable language constructs such as arithmetic expressions, assignments, `for range(...)` loops, conditionals, function definitions, array-style indexing, and simple calls. The resulting C++ code is then analyzed by the same loop extraction path used for native C/C++ input. This design avoids building two separate parallelization pipelines and ensures that both Python and C++ inputs are judged using the same loop features and safety checks.

After normalization, the module extracts loop information using native Clang AST bindings. Each loop is represented by its source code and source-line range. The Python orchestration layer then computes a fixed loop-feature representation, uses a classifier endpoint to predict whether the loop is a parallelization candidate, and validates the decision with rule-based checks. If the loop is safe, the module injects an OpenMP directive such as `#pragma omp parallel for`. For reduction-like loops, it can also emit a reduction clause when the reduction variable is valid.

The module has a fail-safe behavior. If the input cannot be converted, analyzed, classified, compiled, or loaded as a native extension, the system falls back to the original Python function or leaves the code unmodified. This is important because parallelization is an optimization, not a reason to change the semantic meaning of the user's program.

4.4.2 Modular Decomposition

The Code Parallelizer module is decomposed into six cooperating components: input normalization, Python-to-C++ conversion, Clang-based loop extraction, feature extraction and vectorization, classification with rule-based validation, and OpenMP/native-extension generation.

Table 4.3: Main components of the Code Parallelizer module

Component	Responsibility
<code>parallelizer.py</code>	Main orchestration layer. It receives strings, files, custom functions, and custom transformer instances; manages caches; extracts loop features; requests classifier predictions; applies pragmas; and falls back safely when optimization fails.
<code>py2cpp_converter.py</code>	Converts a restricted Python AST subset into C++ code. It handles constants, arithmetic, boolean logic, comparisons, assignments, augmented assignments, if statements, functions, and <code>for range(...)</code> loops.
<code>cpp_compiler.py</code>	Wraps generated C++ into a PyBind11/NumPy module, chooses compiler flags, adds <code>-fopenmp</code> only when OpenMP pragmas are present, builds a shared extension, and loads the compiled function.
<code>src/utils/code_parallelizer.py</code>	Native C++ bridge for loop extraction. It runs a Clang frontend action on source code and serializes extracted loop information to JSON when needed.
<code>include/ast/</code> and related C++ files	Clang AST consumer and loop visitor logic used to identify loop constructs and their source ranges.
Classifier service	Receives the fixed-size loop embedding and predicts one of the supported classes: no parallelization, parallel for, or reduction.

Figure 4.3 summarizes the complete data flow. The important point is that all supported inputs eventually converge to the same C++ loop-analysis path. Only callable Python inputs continue beyond source-code rewriting into PyBind11 extension generation.

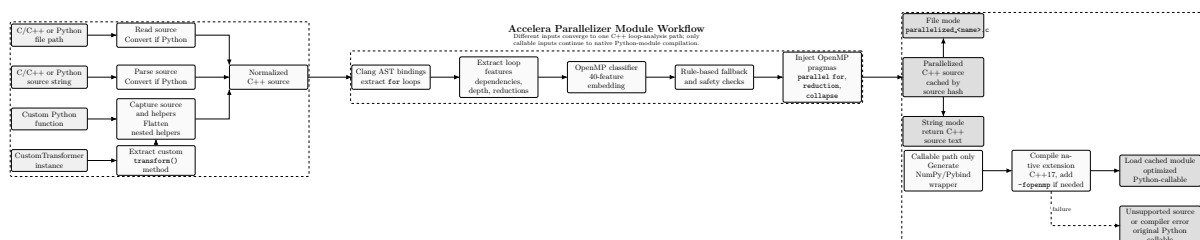


Figure 4.3: Code Parallelizer module workflow

The feature extraction stage combines syntactic and semantic indicators. It looks for reduction patterns, array reads and writes, loop-carried dependencies, indirect memory access, early exits, nested loop depth, branch count, function call count, arithmetic operation count, memory operation count, and whether the loop appears

vectorizable. These features are converted into a 40-dimensional numeric vector before being sent to the classifier. The classifier result is then combined with deterministic rules. For example, even if the classifier returns `none`, the rule layer may still recognize a scalar reduction, an independent array-write loop, or a directly nested loop pattern that can be handled safely.

Algorithm 4.3: Loop classification and pragma insertion

Input: source program P

Output: source program P' with safe OpenMP pragmas

```

1: if  $P$  is Python source then
2:     convert the supported Python subset into C++
3: end if
4: Extract for-loops using the Clang AST frontend
5: for each extracted loop  $L$  do
6:     compute structural, memory, dependency, and reduction features
7:     send the 40-feature vector to the classifier service
8:     combine the predicted class with rule-based safety checks
9:     if the resolved class is parallel_for then
10:         insert #pragma omp parallel for
11:     else if the resolved class is reduction then
12:         insert #pragma omp parallel for reduction(...)
13:     else
14:         leave the loop unchanged
15:     end if
16: end for
17: Cache and return the transformed source code

```

The compilation path is used when the input is a Python callable. The parallelizer extracts or receives the function source, converts it to C++, parallelizes the C++ loop body, wraps the result in a PyBind11 module, compiles it as a shared native extension, loads the extension, and returns the optimized function. Caching is used at both the transformed-source level and the compiled method level. The cache key includes the compilation cache version, the function name, the compiler optimization level, the extension suffix, and the source code. This prevents recompiling the same function repeatedly.

4.4.3 Design Constraints

The most important design constraint of the Code Parallelizer is correctness. OpenMP pragmas can improve performance, but an incorrect pragma can produce wrong numerical results. Therefore, classifier output is not applied blindly. The module applies additional safety rules to avoid unsafe transformations when there are loop-carried dependencies, indirect accesses, early exits, side-effect function calls, or invalid reduction variables. Inner loops are also skipped when an outer selected loop already covers their range, preventing nested duplicate pragma insertion.

The second constraint is the restricted Python subset. Full Python semantics are too dynamic to translate safely into C++ using a lightweight source converter. The converter therefore rejects unsupported syntax such as decorators, slicing, chained comparisons, unsupported operators, non-range loop iterators, multi-target assignment, complex dynamic dispatch, and unsupported expression forms. This restriction is not a weakness in the design; it is a correctness boundary. The system optimizes code only when it can produce predictable C++.

The third constraint is compiler and platform compatibility. Native callable optimization depends on a working C++ compiler, Python include paths, PyBind11 headers, NumPy-compatible array wrapping, and platform-specific shared extension suffixes. The compiler path uses C++17 and adds `-fopenmp` only when the parallelized code actually contains OpenMP pragmas. On non-Windows platforms it also uses position-independent code flags suitable for shared libraries. If compilation fails, the original Python callable is returned.

The fourth constraint is input and output compatibility. The current compiled callable path is oriented toward functions with one named input parameter and NumPy-like two-dimensional numeric data. The wrapper rewrites array access to PyBind11 unchecked NumPy access and exposes the compiled function back to Python. This makes the generated extension useful for numeric preprocessing loops, but it also means that arbitrary Python functions with complex object structures are outside the supported optimization target.

The fifth constraint is reproducibility and caching. Because native compilation can be expensive, the module caches processed C++ code and compiled functions using source hashes. The cache must include enough information to avoid stale or incorrect reuse when compiler options, cache version, function name, source code, or extension suffix changes. This design improves repeated execution time while keeping cache invalidation explicit.

4.4.4 Other Description of Module

The Code Parallelizer is directly integrated with Accelera Pipe. When a preprocessing node is added, the pipeline calls the parallelizer before storing the preprocessing object. If the object is a custom Python function, it is wrapped as a source-backed function whose runtime callable can be replaced by the compiled implementation. If the object is a custom transformer instance, the parallelizer attempts to optimize its `transform` method and attach the optimized method back to the instance. If the object is a library transformer or an unsupported callable, it is returned unchanged.

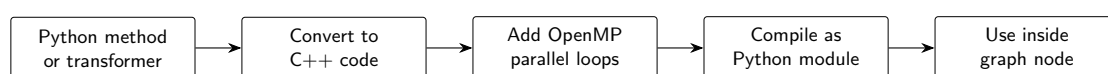


Figure 4.4: Integration flow for compiled preprocessing inside Accelera Pipe

Scientifically, the final design uses a hybrid approach rather than a purely generative code model. An earlier generator-based design attempted to emit parallelized code using a neural sequence model. That approach was rejected because the dataset size was not sufficient for reliable sequence generation and because generated synthetic samples risked introducing distribution bias. The current design is more controlled: the classifier only predicts the type of parallelization opportunity, while deterministic compiler-like rules decide how and whether the pragma is inserted. This separation makes the system easier to inspect, easier to debug, and safer for user code.

Overall, the Code Parallelizer acts as an optimization layer around user-defined numeric loops. It does not attempt to parallelize all Python programs. Instead, it targets the subset where static analysis, feature extraction, classifier prediction, rule validation, and native compilation can cooperate safely. This fits the broader Accelera design philosophy: keep the Python API simple, but move performance-sensitive and structurally analyzable work into optimized native execution when the system can do so without changing program semantics.

4.5 AutoPreprocessing Module

The AutoPreprocessing module is responsible for automating the data understanding, exploratory data analysis, and preprocessing stages of the data science workflow. The module was designed to reduce repeated manual work and to prepare different types of datasets in a consistent way before model training.

4.5.1 Functional Description

The main function of this module is to receive a dataset and user configuration, then return processed data that is ready for machine learning or deep learning models. The module supports tabular, text, and image datasets.

For tabular datasets, the module supports both classification and regression tasks. It analyzes the dataset, removes duplicate records, splits the data into training and validation sets, detects column types, handles missing values, encodes categorical features, scales numerical features, applies target preprocessing, performs feature selection, and saves the preprocessing objects needed later for test data.

For text datasets, the module supports classification tasks. It cleans the text, tokenizes it, applies text preprocessing steps, and converts the text into a numerical representation suitable for machine learning models. For image datasets, the module supports image classification by reading images from class folders, validating image files, resizing images, normalizing pixel values, and preparing the data loaders needed for training.

The module also generates a report when this option is enabled. The report summarizes information about the dataset and the preprocessing steps applied. For example,

in tabular data, it includes dataset shape, column types, missing values, duplicate analysis, generated graphs, removed columns, and previews of the processed data.

4.5.2 Modular Decomposition

The AutoPreprocessing module is decomposed into smaller components according to the dataset type and the stage of preprocessing.

4.5.2.1 Input and Configuration Validation

This component receives the user inputs, such as the dataset, target column, problem type, output folder, validation size, random state, missing-value threshold, cardinality threshold, and report flag. It validates these values before running the pipeline to prevent invalid configurations, such as a wrong target column, unsupported problem type, or invalid threshold values.

4.5.2.2 Data Understanding and Reporting

This component collects the first information about the dataset before preprocessing. It stores dataset previews, column data types, missing values, duplicate count, dataset shape, numerical statistics, and categorical statistics. These results are saved in a report structure and later used to generate the final preprocessing report.

4.5.2.3 Tabular Preprocessing

This component handles structured data organized in rows and columns. It first removes duplicates, then splits the dataset into training and validation sets. After that, it detects feature types such as binary, ordinal, numerical, continuous, low-cardinality categorical, high-cardinality categorical, and unsupported columns.

Each feature type has its own preprocessing pipeline. Numerical and continuous features use median imputation followed by robust scaling. Binary features use most-frequent imputation followed by ordinal encoding. Low-cardinality categorical features use most-frequent imputation followed by binary encoding. High-cardinality categorical features use most-frequent imputation followed by frequency encoding. Unsupported columns and columns above the missing-value threshold are removed.

4.5.2.4 Target Preprocessing

This component prepares the target column. For classification, missing target values are filled using the most frequent class, then the target is transformed using label encoding. For regression, missing target values are filled using the median, then robust scaling is applied to the target values.

4.5.2.5 Feature Selection

After preprocessing the input features, mutual information is used to select the most informative features. For classification, mutual information classification is used. For regression, mutual information regression is used. Only features with scores greater than or equal to the selected threshold are kept.

4.5.2.6 Text Preprocessing

This component handles text classification datasets. It validates the text configuration, cleans the text, applies tokenization and text normalization, and converts the processed text into numerical features. The generated preprocessing objects are saved so the same transformations can be applied to testing data.

4.5.2.7 Image Preprocessing

This component handles image classification datasets. Images are collected from class folders and assigned labels based on their folder names. The system checks whether images can be loaded correctly, removes invalid images, resizes valid images, normalizes pixel values, and applies optional augmentation such as flipping, rotation, brightness adjustment, and contrast adjustment.

4.5.2.8 Artifact Saving

This component saves the objects required to reproduce the same preprocessing steps later. Examples include selected columns, dropped columns, feature names, target preprocessor, training preprocessor, class mappings, and report data. This makes the preprocessing pipeline reusable for validation, testing, or future prediction.

4.5.3 Design Constraints

Several constraints affected the design of the AutoPreprocessing module:

- The module must avoid data leakage. For this reason, preprocessing objects are fitted only on the training data and then applied to validation or testing data.
- The module must support different data types. Therefore, separate preprocessing paths were designed for tabular, text, and image datasets.
- The tabular path assumes that the data is structured in rows and columns and that the task is either classification or regression.
- The text path assumes that the dataset contains a text column and a target label for a text classification task.
- The image path assumes that image classification data follows a folder-based class structure, where each folder represents one class.

- The module must keep preprocessing reproducible. Therefore, fitted preprocessing objects and selected features are saved for later use.
- The module must be general enough for different datasets, so the design uses rule-based decisions instead of manual preprocessing code for each dataset.

4.5.4 Other Description of the Module

The AutoPreprocessing module is not only designed to transform data. It is also designed to help the user understand the dataset before and after preprocessing. This is achieved through the generated report and visualizations. For tabular data, the report can show missing-value statistics, duplicate information, feature distributions, target distribution, correlation matrix, removed columns, preprocessing decisions, and previews of the final processed training and validation data.

Another important design point is that the module separates training preprocessing from testing preprocessing. During training, the module fits the preprocessing objects and saves them. During testing, the saved objects are loaded and applied directly, which ensures that new data is transformed in the same way as the training data.

4.6 AutoML Module

4.7 Deployment Module

4.8 Benchmark Platform Module

The Benchmark Platform module is designed to provide a shared environment where users can create machine learning benchmarks, submit predicted results, and compare submissions using selected evaluation metrics. The module includes a backend API, database schemas, Python validation and scoring scripts, and a frontend interface for users and admins.

4.8.1 Functional Description

The main function of this module is to support fair comparison between different machine learning solutions. A user can create a benchmark by providing the benchmark title, description, problem type, target column, dataset link, test dataset link, ground-truth target link, and evaluation metric. The system validates the provided links and checks that the test set does not contain the target column while the target file contains the expected target column.

After a benchmark is created, other users can submit their prediction file and repository link. The system validates the submitted prediction file, calculates the score using the selected Scikit-learn metric, stores the submission, and displays submissions on a leaderboard. The leaderboard is ordered based on the metric configuration, where some metrics are better when their value is higher and others are better when their value is lower.

4.8.2 Modular Decomposition

The Benchmark Platform is decomposed into several smaller components:

4.8.2.1 Benchmark Management

This component handles benchmark creation, listing, filtering, viewing, and deletion. It stores benchmark information such as title, description, problem type, dataset links, target column, selected metric, metric parameters, creation date, and creator.

4.8.2.2 Metric Management

This component allows admin users to define evaluation metrics. Each metric contains a display name, the corresponding Scikit-learn metric function name, needed parameters, problem type, and whether a higher or lower score is better. Metrics are linked to benchmarks and used during submission scoring.

4.8.2.3 Submission Management

This component allows users to submit prediction files for a specific benchmark. Each submission stores the benchmark id, submitting user, repository link, prediction file link, score, and submission date. Users can also update their existing prediction file or delete their submission if they have permission.

4.8.2.4 Validation and Scoring Scripts

The backend calls Python scripts to validate benchmark files and score submissions. During benchmark creation, the validation script checks that the test file does not include the target column and that the target file contains the required target column. During submission, the scoring script loads the ground-truth file and the submitted prediction file, checks their length and target column, then computes the selected Scikit-learn metric.

4.8.2.5 Leaderboard Interface

The frontend displays benchmark submissions in a leaderboard. It shows the submitter information, repository link, score, and submission date. The sorting direction depends

on the selected metric, so the platform can support both metrics where higher is better and metrics where lower is better.

4.8.3 Design Constraints

Several constraints affected the design of the Benchmark Platform:

- The module currently supports classification and regression problem types only.
- Dataset, test, target, and prediction files are provided using links, mainly Google Drive file links, to reduce server storage requirements.
- The test dataset must not contain the target column. This prevents data leakage and makes the evaluation fair.
- The target file and submitted prediction file must contain the selected target column and must have the same number of rows.
- Evaluation metrics must be defined using valid Scikit-learn metric names because the scoring script calls Scikit-learn metrics dynamically.
- Only authenticated users can create benchmarks or submit predictions. Admin permission is required for metric creation.
- A user is allowed to create only one active submission for the same benchmark. If the user wants to change the result, the existing submission is updated instead of creating duplicate submissions.

4.8.4 Other Description of the Module

The Benchmark Platform separates benchmark creation from submission evaluation. This makes the workflow clearer: first, the benchmark creator defines the task and evaluation rules; then, users submit predictions under the same conditions. This separation helps ensure that all users are evaluated using the same ground-truth file and the same metric configuration.

Another important design point is the use of Python scripts for file validation and metric scoring. The backend is implemented using Node.js and Express, while the scoring process uses Python because it depends on Scikit-learn metrics and dataset loading utilities already used in the project. This design allows the web platform to manage users, benchmarks, and submissions while Python handles the machine learning evaluation part.

Chapter 5: System Testing and Verification

5.1 Testing Setup

The available experiments use identical train, validation, and test splits when comparing candidate pipelines. Timing, memory, output equivalence, and predictive accuracy are reported according to the module under test.

5.2 Testing Plan and Strategy

Module tests validate graph correctness, pipeline persistence, converter support, loop transformation, and output equivalence. Integration tests evaluate the end-to-end path from custom preprocessing code through native compilation and graph execution.

5.3 Accelera Pipe Module

The Accelera Pipe experiments compare three implementations under the same train, validation, and test split. The first implementation is a manually written scikit-learn loop over the candidate pipelines. The second uses `sklearn.pipeline.Pipeline` to represent the same candidates. The third uses Accelera Pipe with graph branches. Each candidate is selected using validation accuracy, and the final selected path is evaluated on the test set. This protocol checks whether the graph execution improves latency without changing the selected model or the measured predictive accuracy.

Table 5.1: Accelera Pipe latency scaling in seconds

Samples	sklearn	sklearn Pipeline	Accelera Pipe
1,000	0.06	0.05	0.34
5,000	2.47	2.12	2.02
10,000	2.48	2.48	1.79
50,000	23.96	23.25	14.81
100,000	77.79	77.10	67.04
250,000	803.56	803.01	494.64

Table 5.1 shows the main performance trend without compressing the font. At very small sizes, Accelera Pipe is slower because the native graph setup, branch creation, and Python/C++ boundary cost dominate the actual model computation. Once the dataset grows, those fixed costs become small compared with fitting multiple candidate

branches. At 50,000 samples, Accelera Pipe reduces runtime from 23.96 seconds to 14.81 seconds. At 250,000 samples, it reduces runtime from about 803 seconds to about 495 seconds, saving more than five minutes while selecting the same candidate.

Table 5.2: Accelera Pipe RSS memory scaling in MB

Samples	sklearn	sklearn Pipeline	Accelera Pipe
1,000	1.62	0.16	19.32
5,000	3.62	1.09	21.62
10,000	11.47	2.19	7.25
50,000	126.95	9.75	209.25
100,000	139.70	47.65	300.93
250,000	303.95	28.52	594.86

Table 5.2 shows the current cost of graph-level parallelism. Accelera Pipe uses more RSS memory because multiple branch states and intermediate arrays can exist at the same time. The memory optimizations in the methodology section, especially consumer-count based release, duplicate first-node sharing, and guarded preprocessing copies, target exactly this issue.

Table 5.3: Selected candidate and accuracy across sample sizes

Samples	Selected candidate	Validation accuracy	Test accuracy
1,000	MinMaxScaler → SVC	0.7900	0.8250
5,000	MinMaxScaler → SVC	0.9160	0.9100
10,000	MinMaxScaler → SVC	0.9385	0.9260
50,000	MinMaxScaler → SVC	0.9565	0.9598
100,000	StandardScaler → SVC	0.9675	0.9709
250,000	StandardScaler → SVC	0.9774	0.9768

Table 5.3 confirms that the speedup does not come from changing the search space. Accelera Pipe evaluates the same candidate pipelines and reports the same validation and test behavior. The selected preprocessing strategy changes from Min-Max scaling to standard scaling only when the larger dataset makes StandardScaler with SVC the best validation candidate. This is expected because the candidate set and selection rule are fixed while the data distribution becomes better represented at larger sizes.

Table 5.4: Largest Accelera Pipe comparison

Implementation	Time (s)	RSS MB	VMS MB	Test accuracy
sklearn manual loop	803.56	303.95	315.41	0.9768
sklearn Pipeline	803.01	28.52	28.61	0.9768
Accelera Pipe	494.64	594.86	787.47	0.9768

Table 5.4 isolates the largest run: 150,000 training samples, 50,000 validation samples, and 50,000 test samples. Accelera Pipe keeps exactly the same selected path, StandardScaler followed by SVC, and the same test accuracy. The runtime drop is 308.91 seconds relative to the manual scikit-learn loop and 308.36 seconds relative to scikit-learn Pipeline. This result is important because it demonstrates the value of graph scheduling on realistic branch-heavy workloads. The memory columns also show the next engineering target: scikit-learn Pipeline is extremely memory efficient, while Accelera Pipe trades memory for parallel branch execution and native graph state.

5.4 Code Parallelizer Module

The OpenMP classifier was evaluated as a three-class loop classifier. The labels are `none`, `parallel_for`, and `reduction`. The classification report shows that the model is balanced enough to guide the rule layer, while the final pragma decision remains protected by dependency checks.

Table 5.5: OpenMP classifier report

Class	Precision	Recall	F1-score	Support
none	0.82	0.85	0.84	3048
parallel_for	0.86	0.81	0.83	2696
reduction	0.79	0.82	0.80	775
accuracy			0.83	6519
macro avg	0.82	0.83	0.83	6519
weighted avg	0.83	0.83	0.83	6519

Table 5.5 should be read as a guidance-quality result, not as a proof that every predicted loop is safe. The classifier is responsible for identifying likely parallelization opportunities. The rule layer is responsible for rejecting unsafe or unsupported cases. This two-stage design is useful because source code parallelization has correctness constraints that pure statistical classification should not decide alone.

Table 5.6: Hard parallelizer evaluation

File	Baseline ms	Parallel ms	Speedup	Output match
hard_eval_000.cpp	1786.28	195.47	9.138×	true
hard_eval_001.cpp	63.98	31.47	2.033×	true
hard_eval_002.py	42.66	7.67	5.563×	true

Table 5.6 evaluates harder programs from `data/hard_test_files`. The benchmark compiles and runs both the baseline and the generated parallelized code, then compares their outputs. The first file obtains the largest improvement because its loop body has enough work to amortize OpenMP thread overhead. The second file still improves, but the smaller absolute runtime limits the maximum speedup. The Python-origin file demonstrates the full converter path: Python subset to C++, then loop extraction, then pragma insertion. Across the three files, the evaluation extracted 11 loops, generated 8 pragmas, matched all 8 expected pragmas, reached average similarity 1.0, and obtained an average speedup of 5.578×

Table 5.7: Linux parallelizer and Accelera Pipe integration benchmark

Mode	Samples	Time (s)
Python custom preprocessing function	500,000	6.83
End-to-end optimized run, first measured process	500,000	5.88
End-to-end optimized run, warmed method and module cache	500,000	0.08

Table 5.7 reports the Linux direct `examples/parallel_accpipe.py` run after normalizing the example execution directory to the repository root. The Python implementation takes 6.83 seconds on 500,000 samples. The first optimized process in the measurement sequence takes 5.88 seconds because it still pays process-local setup and cache lookup costs. The immediate repeated run takes 0.08 seconds after the Python-method cache and compiled-module cache are both warm. The validation in the example confirmed that the optimized output matched the Python output. This result shows why the integration needs two cache layers: one layer avoids repeating Python-to-C++ conversion and OpenMP insertion, while the lower compiled-module cache avoids recompiling the pybind extension.

Table 5.8: Linux direct example-script verification runs

Example	Baseline time (s)	Optimized time (s)	Validation
accpipe_demo	2.48	1.79	same accuracy, 0.9260
parallel_accpipe, run 1	6.83	5.88	outputs match
parallel_accpipe, run 2	6.83	0.08	outputs match
code_parallelizer, run 1	5.81	0.014	outputs match
code_parallelizer, run 2	6.05	0.015	outputs match
save_load, run 1	1.89	2.69 / 2.65	same accuracy, 0.268
save_load, run 2	2.53	2.20 / 2.14	same accuracy, 0.268

Table 5.8 summarizes the Linux examples invoked by `examples/Makefile`, but each file was run directly rather than through Make. Cache-sensitive files were repeated. The standalone `code_parallelizer_demo.py` result measures Python script execution against the generated OpenMP executable for `hard_eval_002.py`; the C path also reported about 0.26–0.27 seconds of compilation time, outside the listed executable runtime. The save/load example verifies persistence rather than pure acceleration: both sklearn and Accelera load persisted state and produce the same accuracy, while the second Accelera run is slightly faster than the sklearn baseline.

Table 5.9: Windows direct example correctness and path timing

Example	Run/cache state	Baseline (s)	Accelera/C path (s)	Validation
accpipe_demo	normal	1.75 / 1.78	1.03	same test accuracy, 0.9260
parallel_accpipe	isolated cache, run 1	6.66	6.00	outputs match
parallel_accpipe	isolated cache, run 2	6.58	0.09	outputs match
code_parallelizer	isolated cache, run 1	5.59	1.04	outputs match
code_parallelizer	isolated cache, run 2	5.61	1.05	outputs match
save_load	normal	0.57	0.51 / 0.53	same accuracy, 0.268

Table 5.10: Windows compilation and process overhead

Example	Run/cache state	Compile/link (s)	Wall time (s)	Interpretation
accpipe_demo	normal	—	6.53	graph example startup
parallel_accpipe	isolated cache, run 1	included	14.37	cold compile path
parallel_accpipe	isolated cache, run 2	cached	8.33	warm cache path
code_parallelizer	isolated cache, run 1	0.87	8.74	temporary executable
code_parallelizer	isolated cache, run 2	1.04	9.00	temporary executable
save_load	normal	—	2.71	persistence example

Tables 5.9 and 5.10 report the same Makefile example set on Windows. The first table keeps the user-visible comparison: baseline runtime, optimized path runtime, and validation result. The second table separates compilation and process-level overhead so the timing table remains readable. The examples were run one by one

in Makefile order. The cache-sensitive examples were then repeated under an isolated Accelera cache so that the first run represents cold cache behavior and the second run represents warm cache behavior. The strongest cache effect appears in `parallel_accpipe.py`: the cold end-to-end Accelera path takes 6.00 seconds because it must convert the function, select loops, compile a pybind extension, link it, and load it into Python. The immediate warm-cache run takes 0.09 seconds because the processed-code cache, method cache, and compiled-module cache avoid repeating those setup stages.

The Windows `code_parallelizer_demo.py` runs show a different limitation. The generated C/OpenMP executable is compiled into a temporary directory on every run, so the second run does not reuse the final linked program. The measured C executable subprocess takes about 1.04 seconds, while the instrumented main-body timer inside the generated program reports only about 0.024–0.025 seconds. This gap is Windows process startup, OpenMP runtime initialization, dynamic library loading, and executable loading overhead. The compile/link step itself adds about 0.87–1.04 seconds. On Linux, the same benchmark has lower launch and linking overhead in the reported measurements, so executable runtime is closer to the timed main-body work. For this reason, Windows results should be interpreted as end-to-end toolchain latency results, not only as loop-kernel speed results.

5.5 AutoPreprocessing Module

5.5.1 Experimental Setup

Experimental Environment All experiments were implemented in Python using Pandas, NumPy, and Scikit-learn for tabular data processing and model evaluation. AutoCleanML was used as the baseline preprocessing framework, while Accelera was used as the proposed automated preprocessing system. The experiments were executed under identical conditions.

Datasets A total of 16 tabular datasets were used, collected from Kaggle. The datasets include both classification and regression tasks with varying sizes and feature complexities.

Table 5.11: Summary of Datasets Used in Experiments

Dataset	Shape	Task Type
startup_founder_burnout_2026	(50000, 29)	Classification
healthy_diet_calorie_intake	(6000, 15)	Classification
ai_student	(50000, 16)	Classification
gaming_addiction	(250, 49)	Classification
Teen_Mental_Health_Dataset	(1200, 13)	Classification
student_exam_performance_dataset	(10000, 17)	Classification
heart	(302, 14)	Classification
Dataset_spine	(310, 7)	Classification
diabetes_dataset	(100000, 31)	Classification
student_placement_synthetic	(100000, 18)	Classification
Titanic-Dataset	(891, 12)	Classification
customer_purchase_data	(1388, 9)	Classification
global_ai_jobs	(90000, 35)	Regression
CarPrice_Assignment	(205, 26)	Regression
Concrete_Data_Yeh	(1005, 9)	Regression
Housing	(545, 13)	Regression

Compared Frameworks Two preprocessing tools were evaluated:

AutoCleanML: A fully automated data cleaning AutoCleanML Python library, that performs missing value handling, encoding, and feature preparation.

Accelera: A proposed automated preprocessing tools supporting tabular, image, and text data, with additional reporting and visualization capabilities.

Preprocessing Configuration Identical experimental settings were used across both tools. A validation size of 20% and a random state of 42 were applied for all datasets. For classification tasks, stratified sampling was used to preserve class distribution.

Apart from specifying the target column and excluding unnecessary columns when required, all preprocessing parameters were kept at their default values for both frameworks.

Results and Discussion

Classification Results The performance of both frameworks was evaluated on multiple classification datasets using F1-macro score as the evaluation metric.

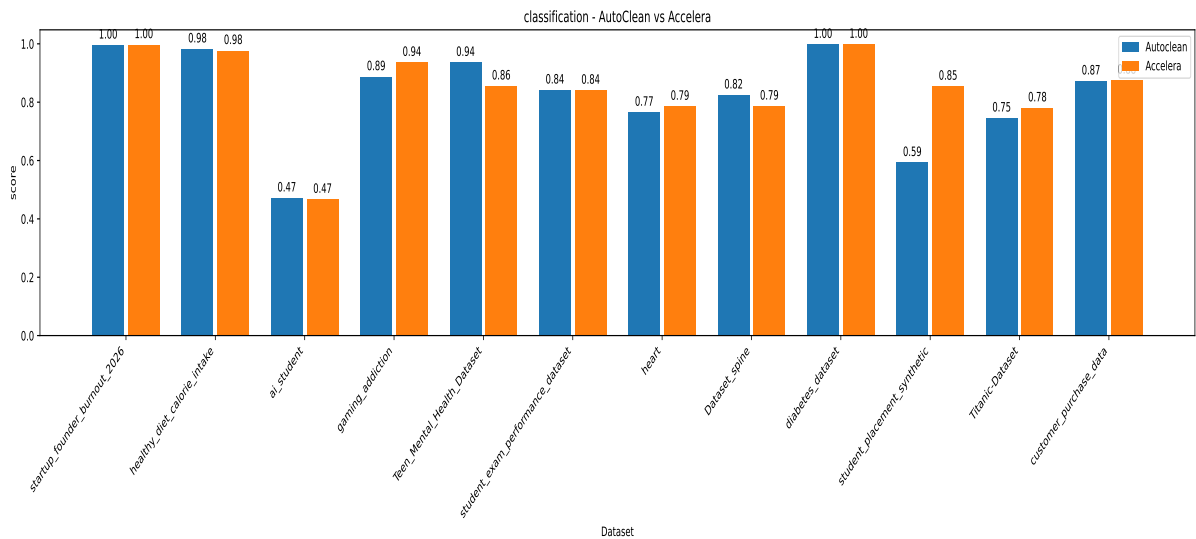


Figure 5.1: Classification Performance Comparison (F1-Macro Score)

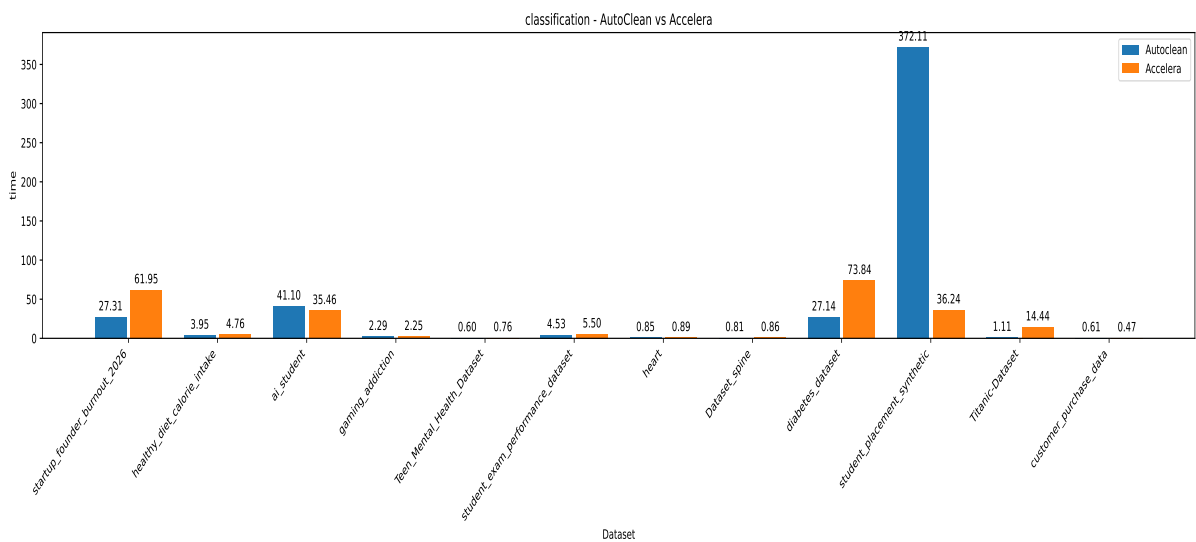


Figure 5.2: Classification Preprocessing Time Comparison

Regression Results Regression performance was evaluated using the R^2 score.

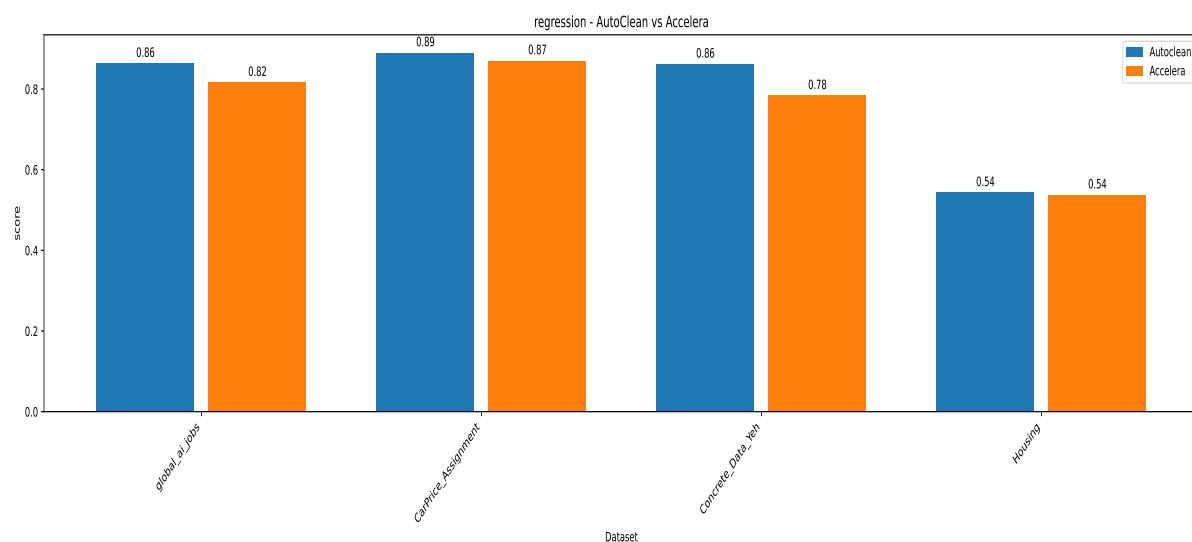


Figure 5.3: Regression Performance Comparison (R^2 Score)

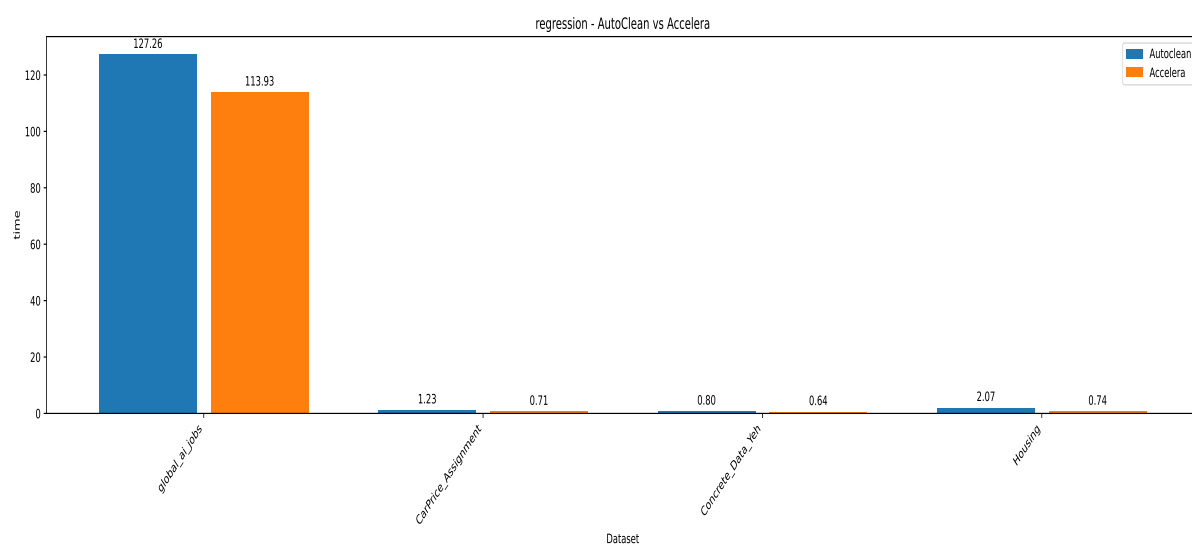


Figure 5.4: Regression Preprocessing Time Comparison

5.6 AutoML Module

5.7 Deployment Module

5.8 Benchmark Platform Module

Chapter 6: Conclusions and Future Work

6.1 Faced Challenges

The main implementation challenges were preserving correctness while sharing branch computations, releasing intermediate arrays only after their final consumer, and limiting generated OpenMP code to loops that pass dependency and conversion checks. The experiments also identify memory usage during branch-level parallel execution as the main remaining graph-backend challenge.

6.2 Gained Experience

6.3 Conclusions

Accelerera Pipe provides a graph-based execution model for branch-heavy machine-learning pipelines and supports fitted executed graphs. The Code Parallelizer provides a source-to-source path from supported Python or C++ loops to OpenMP-annotated C++ using converter rules, loop feature extraction, classifier guidance, and safety checks. The available experiments show runtime improvements on large branch-selection workloads while preserving selected models, accuracy, and output equivalence.

6.4 Future Work

Future work should continue reducing graph memory consumption while preserving parallel branch execution, and should extend safe converter and loop-analysis coverage without weakening correctness checks.

References

Bibliography

- [1] "Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices." Available at: <https://www.proquest.com/openview/17dfa731b6147c8939e3c5cc784897ce/1?pq-origsite=gscholar&cbl=5046920>

Chapter A: Development Platforms and Tools

The implemented modules use Python, C++, PyBind11, CMake, Clang-based AST analysis, OpenMP, NumPy, and Scikit-learn. The available experiments include Linux and Windows execution paths.

Chapter B: Use Cases

Chapter C: User Guide

Chapter D: Code Documentation

Chapter E: Feasibility Study